RTL-to-GDSII Design and Implementation of a 4-Bit Registered Array Multiplier in SkyWater 130 nm CMOS

Manikya Pant
Roll Number: 2023112024
Digital VLSI Design, IIIT Hyderabad
manikya.pant@research.iiit.ac.in
GitHub: https://github.com/ManikyaPant/DVD_assignment

Aryan Agrawal
Roll Number: 2023102050
Digital VLSI Design, IIIT Hyderabad
aryan.agrawal@students.iiit.ac.in
GitHub: https://github.com/ManikyaPant/DVD_assignment

***Abstract*—This report presents the complete end-to-end open-source RTL-to-GDSII implementation of a 4-bit unsigned registered array multiplier (`mult_array`) in the SkyWater 130 nm CMOS process (`sky130A`, standard-cell library `sky130_fd_sc_hd`). The architecture comprises 16 two-input partial-product AND gates, 12 full adders arranged in three 4-bit ripple-carry rows, 8 input flip-flops, and 8 output flip-flops, providing a 2-clock-cycle latency and 1-result/cycle throughput. Exhaustive self-checking verification in Icarus Verilog across all 256 input vectors passes with 0 errors in both behavioral and gate-level simulation, while an error-injection test (`sum = a ^ b`) catches 95 errors. Yosys synthesis maps the design to 67 cells (746.966 μm^2, 53.60% flip-flop area). Pre-layout OpenSTA analysis yields $T_{\min} = 2.604$ ns ($F_{\max} = 384.02$ MHz) along an 8-cell combinational critical path from `b_reg[0]` to `p[6]`. Physical implementation in LibreLane/OpenROAD on a 150 μm $\times$ 150 μm die converges in 8 clock-period iterations at $T_{\text{clk}} = 7.0$ ns ($F_{\max} = 153.01$ MHz worst-case `max_ss_100C_1v60`, 316.34 MHz nominal `nom_tt_025C_1v80`) with 0 DRC and 0 LVS violations.**

***Index Terms*—Array multiplier, RTL-to-GDSII, SkyWater SKY130, Yosys, OpenSTA, LibreLane, OpenROAD, DRC, LVS.**

I. Introduction

Binary multiplication is a fundamental arithmetic operation in digital signal processors, general-purpose microprocessors, and hardware accelerators. This work implements and characterizes a 4-bit $\times$ 4-bit unsigned registered array multiplier (`mult_array`) through a complete open-source RTL-to-GDSII flow inside the IIC-OSIC-TOOLS environment targeting the SkyWater 130 nm high-density standard-cell library (`sky130_fd_sc_hd`, nominal corner `tt_025C_1v80` at $V_{\text{DD}} = 1.80$ V, $T = 25\,^{\circ}$C). All design files, scripts, and reports are available in the project GitHub repository at https://github.com/ManikyaPant/DVD_assignment.

Given two 4-bit unsigned operands $A = a_3a_2a_1a_0 \in [0,15]$ and $B = b_3b_2b_1b_0 \in [0,15]$, the multiplier computes the 8-bit unsigned product $P = p_7p_6p_5p_4p_3p_2p_1p_0 = A \times B \in [0,225]$ via binary shift-and-add:

$$P = \sum_{i=0}^{3} (A \cdot b_i)\, 2^i = \sum_{i=0}^{3} \sum_{j=0}^{3} (a_j \wedge b_i)\, 2^{i+j} \tag{1}$$

The combinational core uses **16 two-input AND gates** to form four 4-bit partial products $\text{pp}_i[j] = a_j \wedge b_i$ and **12 1-bit full adders** arranged in three 4-bit ripple-carry adder rows (`row1`, `row2`, `row3`). Registering the two 4-bit inputs (`a_reg[3:0]`, `b_reg[3:0]`) and the 8-bit product (`p[7:0]`) on the rising edge of `clk` with active-low asynchronous reset (`rst_n`) uses **16 flip-flops**, resulting in a **2-clock-cycle latency** and a **throughput of 1 result per clock cycle**.

II. Design and Verification (Part A)

A. RTL Architecture and Exact Connections

The synthesizable RTL is written in two Verilog modules without using the $\star$ operator:

- `rtl/full_adder.v`: A 1-bit combinational full adder with inputs `a`, `b`, `cin` and outputs `sum`, `cout` implemented with `assign` statements: `assign sum = a ^ b ^ cin;` and `assign cout = (a & b) | (b & cin) | (cin & a);`.
- `rtl/mult_array.v`: Registers `a[3:0]` and `b[3:0]` into `a_reg[3:0]` and `b_reg[3:0]` on `posedge clk` (or clears all registers to 0 on `negedge rst_n`). Sixteen AND gates form `pp0 = a_reg & {4{b_reg[0]}}` through `pp3`. Row 0 produces `p[0] = pp0[0]` and passes `r0 = {1'b0, pp0[3:1]}` to Row 1. Rows 1–3 instantiate 12 `full_adder` modules via a `generate` loop (`adder_columns[0..3].row1..3`) with `c1[0] = c2[0] = c3[0] = 1'b0`, passing `r1 = {c1[4], s1[3:1]}` and `r2 = {c2[4], s2[3:1]}` downward and registering `product = {c3[4], s3[3:1], s3[0], s2[0], s1[0], pp0[0]}` into `p[7:0]`.

GitHub Repository: https://github.com/ManikyaPant/DVD_assignment

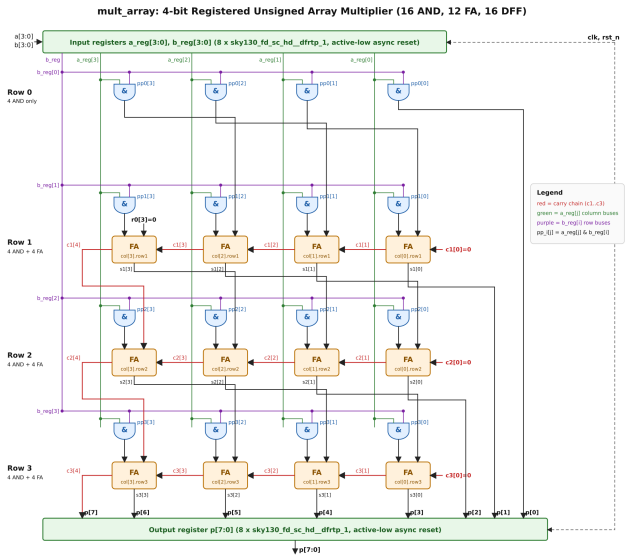


Fig. 1: Structural block diagram of `mult_array` showing the 8-bit input register bank (`a_reg[3:0]`, `b_reg[3:0]`), 16 partial-product AND gates (`pp0`–`pp3`), 12 full adders across three 4-bit ripple-carry rows (`row1`–`row3`), and the 8-bit output register (`p[7:0]`).

Fig. 1 shows the complete structural block diagram of `mult_array`.

B. Hand-Worked Example for Roll Number 2023112024 ($A = 9, B = 7$)

Using Manikya Pant's roll number `2023112024`, the last digit is $d = 4$, giving $A = 4 + 5 = 9 = 1001_2$ (`4'b1001`) and $B = 7 = 0111_2$ (`4'b0111`), with expected product $P = 9 \times 7 = 63 = 00111111_2$ (`8'h3F`). The binary shift-and-add calculation and internal row-by-row hardware trace in `rtl/mult_array.v` are shown below:

Hand-Worked Multiplication ($A = 9, B = 7 \rightarrow P = 63$)

```
        1 0 0 1   A = 9 (roll 2023112024: last digit 4 + 5)
      x 0 1 1 1   B = 7
---------------
        1 0 0 1   pp0 = A AND b0 (b0 = 1) -> p[0]=1, r0=0100
      1 0 0 1     pp1 = A AND b1 (b1 = 1), shifted 1
    1 0 0 1       pp2 = A AND b2 (b2 = 1), shifted 2
  0 0 0 0         pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 0 1 1 1 1 1 1   P = 63 (0x3F = 32 + 16 + 8 + 4 + 2 + 1)

Row 0: pp0 = 4'b1001 -> p[0] = pp0[0] = 1; r0 = {0,pp0[3:1]} = 4'b0100
Row 1: pp1 (1001) + r0 (0100), c1[0]=0 -> s1=1101, c1[4:1]=0000
       -> p[1] = s1[0] = 1; passes r1 = {c1[4], s1[3:1]} = 4'b0110
Row 2: pp2 (1001) + r1 (0110), c2[0]=0 -> s2=1111, c2[4:1]=0000
       -> p[2] = s2[0] = 1; passes r2 = {c2[4], s2[3:1]} = 4'b0111
Row 3: pp3 (0000) + r2 (0111), c3[0]=0 -> s3=0111, c3[4:1]=0000
       -> p[3] = s3[0] = 1; p[7:4] = {c3[4], s3[3:1]} = 4'b0011
Final: product[7:0] = 8'b0011_1111 = 63 (0x3F)
```

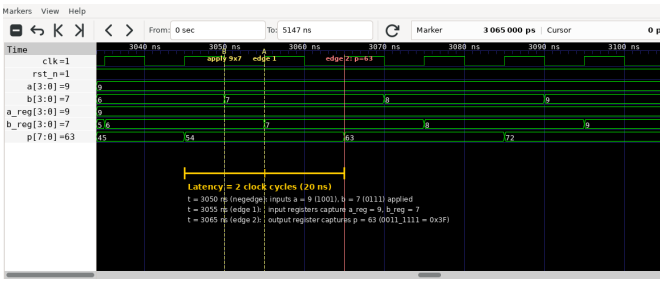


Fig. 2: GTKWave waveform (`mult.vcd`) for $A = 9, B = 7 \rightarrow P = 63$ (`0x3F`) at $T_{\text{clk}} = 10.0$ ns: inputs applied at $t = 3050$ ns, captured into `a_reg`/`b_reg` at rising edge 1 ($t = 3055$ ns), and product $p = 63$ latched into `p[7:0]` at rising edge 2 ($t = 3065$ ns), showing the 2-clock-cycle (20.0 ns) latency.

C. Exhaustive Verification, Passed Run, and Failed Injection

The self-checking testbench (`tb/tb_mult.v`) drives a 10 ns clock (100 MHz), holds `rst_n = 0` for the first 2 rising clock edges, and applies all 256 input combinations $(a, b) \in [0, 15]^2$ on `negedge clk`. For each combination, it waits 2 rising clock edges (2-cycle latency) plus `#2` ns before comparing `p` against `expected = a_value * b_value`, and writes `mult.vcd` (Fig. 2).

- **Passed Run (Task 4, `sum = a ^ b ^ cin`)**: Compiling and running `iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim` checks all 256 vectors and prints `PASS: 256 vectors checked, 0 errors`.
- **Failed Error-Injection Run (Task 6, `sum = a ^ b`)**: Temporarily changing `sum` in `rtl/full_adder.v` to `a ^ b` (omitting `^ cin`) corrupts the sum output whenever any full adder receives `cin` = 1, causing **95 mismatches out of 256 vectors** and printing `FAIL: 256 vectors checked, 95 errors` (complete 95-mismatch log in Appendix A). Restoring `sum = a ^ b ^ cin` restores `PASS`.

Part A Passed Run and Failed Error-Injection Summary Lines

```
# Task 4: Passed RTL Run (unmodified rtl/full_adder.v)
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:75: $finish called at 5146000 (1ps)

# Task 6: Failed Error-Injection Run (sum = a ^ b in rtl/full_adder.v)
VCD info: dumpfile mult.vcd opened for output.
Mismatch: a=3 b=3 expected=9 got=5
Mismatch: a=3 b=6 expected=18 got=10
... [91 intermediate mismatches; full 95-error log in Appendix A] ...
Mismatch: a=15 b=15 expected=225 got=181
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:75: $finish called at 5146000 (1ps)
```

III. Logic Synthesis with Yosys (Part B)

Logic synthesis was executed with Yosys 0.69 (`yosys -c synth/synth.tcl`) using the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`. The script reads `rtl/full_adder.v` and `rtl/mult_array.v`, flattens the hierarchy (`synth`

TABLE I: Complete 23-Cell-Type Breakdown of Synthesized `mult_array` from `synth/mult_array_stat.rpt` (`sky130_fd_sc_hd`).

Cell Type (`sky130_fd_sc_hd__*`)	Count	Area (μm^2)	Cell Type (`sky130_fd_sc_hd__*`)	Count	Area (μm^2)
`dfrtp_1` (Pos-edge DFF w/ async active-low reset)	16	400.384	`a21boi_0` (2-1 AND-OR-Invert w/ inverted B1_N)	1	7.507
`nand2_1` (2-input NAND gate)	12	45.043	`a22oi_1` (2-2 AND-OR-Invert gate)	1	7.507
`xnor2_1` (2-input XNOR gate)	7	61.309	`a32oi_1` (3-2 AND-OR-Invert gate)	1	8.758
`a21oi_1` (2-1 AND-OR-Invert gate)	4	20.019	`and4_1` (4-input AND gate)	1	8.758
`and2_0` (2-input AND gate)	2	12.512	`clkinv_1` (Balanced clock inverter)	1	3.754
`lpflow_isobufsrc_1` (Low-power isolation buffer)	2	12.512	`nand3_1` (3-input NAND gate)	1	5.005
`maj3_1` (3-input majority carry gate)	2	20.019	`nor2_1` (2-input NOR gate)	1	3.754
`nand2b_1` (2-input NAND w/ inverted input A_N)	2	12.512	`o21ba_1` (2-1 OR-AND w/ inverted input B1_N)	1	10.010
`nand4_1` (4-input NAND gate)	2	12.512	`or3_1` (3-input OR gate)	1	6.256
`nor3_1` (3-input NOR gate)	2	10.010	`xor2_1` (2-input XOR gate)	1	8.758
`o21a_1` (2-1 OR-AND gate)	2	15.014			
`o21ai_0` (2-1 OR-AND-Invert gate)	2	10.010			
`xnor3_1` (3-input XNOR gate)	2	45.043	**Total (23 distinct standard-cell types)**	**67**	**746.966**

TABLE II: Part B Logic Synthesis Results Table (`tt_025C_1v80`).

Metric	Value (with Units)
Total number of cells	**67 cells** (16 flip-flops + 51 logic cells)
Number of flip-flops	**16 flip-flops** (`dfrtp_1`: 8 in + 8 out)
Total cell area (μm^2)	**746.966** μm^2 (346.582 μm^2 combinational)
Flip-flop area (μm^2)	**400.384** μm^2 ($16 \times 25.024\,\mu$m^2 per DFF)
Flip-flop area (% of total)	**53.60%** (46.40% combinational logic)
3 most-used cell types	1. `dfrtp_1`: **16 cells** (400.384 μm^2) 2. `nand2_1`: **12 cells** (45.043 μm^2) 3. `xnor2_1`: **7 cells** (61.309 μm^2)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

`-top mult_array -flatten`), maps flip-flops (`dfflibmap`) and combinational logic (`abc`), removes unused wires (`opt_clean -purge`), writes statistics to `synth/mult_array_stat.rpt`, and outputs `synth/mult_array_netlist.v`. Gate-level simulation (`vvp gls`) with `primitives.v` and `sky130_fd_sc_hd.v` (`-DFUNCTIONAL -DUNIT_DELAY=#1`) verified functional equivalence with `PASS: 256 vectors checked, 0 errors`. Table II gives the Part B results table, and Table I lists all 23 standard-cell types.

IV. Static Timing Analysis with OpenSTA (Part C)

Pre-layout STA was performed in OpenSTA 3.1.0 (`sta -exit sta/sta.tcl`) using `sky130_fd_sc_hd__tt_025C_1v80.lib`, 1.0 ns input/output delays relative to `clk`, and ideal interconnect:

1) **Run at** $T = 10.0$ **ns**: Worst setup slack is WNS $= +$**7.396** ns (`worst slack max 7.40`), yielding $T_{\min} = 10.000 - 7.396 =$ **2.604** ns (**2.60** ns rounded to 0.01 ns) and $F_{\max} = 1000/2.604 =$ **384.02** MHz (384.62 MHz at 2.60 ns).
2) **Run at** $T = T_{\min} = 2.60$ **ns**: Worst setup slack is $-$**0.004** ns (`worst slack max -0.00`), confirming slack is within 4 ps of 0 ns.

Table V shows the Part C results table, and Table III traces all cells along the critical path from `sta/mult_array_timing.rpt`.

V. Physical Design: RTL to GDSII (Part D)

Physical implementation from RTL to GDSII was performed in LibreLane (`librelane config.json`) using Yosys, OpenROAD, Magic, KLayout, and Netgen on a fixed $150\,\mu$m $\times$ $150\,\mu$m die (`"FP_SIZING": "absolute", "DIE_AREA": [0, 0, 150, 150]`). Starting from $T_{\text{start}} = \lceil 1.2 \times T_{\min} \rceil_{0.5\text{ ns}} =$ **3.5** ns, `CLOCK_PERIOD` in `pnr/config.json` was increased by 0.5 ns after each setup violation in the worst-case slow-slow corner (`max_ss_100C_1v60` at 1.60 V, 100°C) until Run 8 (7.0 ns) passed all 9 PVT corners with 0 DRC and 0 LVS errors. Table IV records all 8 runs, Table VI summarizes the final signoff metrics from `pnr/metrics.json`, and Fig. 3 shows the KLayout whole-die and zoomed screenshots.

VI. Answers to Questions (Section 10)

1) Logic Cell Count (51 Cells vs. $16+12=28$): Yosys produced $67-16=$ **51** combinational logic cells (346.582 μm^2) rather than $16+12=28$. First, each structural `full_adder` (sum $= a \oplus b \oplus c_{\text{in}}$, $c_{\text{out}} = \text{maj}(a,b,c_{\text{in}})$) is decomposed into multiple primitive standard cells (`xnor2_1`, `xnor3_1`, `maj3_1`, `nand2_1`, AOI/OAI gates) rather than one monolithic cell. Second, the three row-LSB full adders have constant $c_{\text{in}} = 0$, which constant propagation simplifies into 2-input half-adder logic. Third, `synth -flatten` and ABC technology mapping merge and invert shared sub-expressions across the partial-product AND gates and adjacent adder stages (mapping 12 of the 16 AND gates into smaller/faster inverting `nand2_1` and `nand4_1` cells).

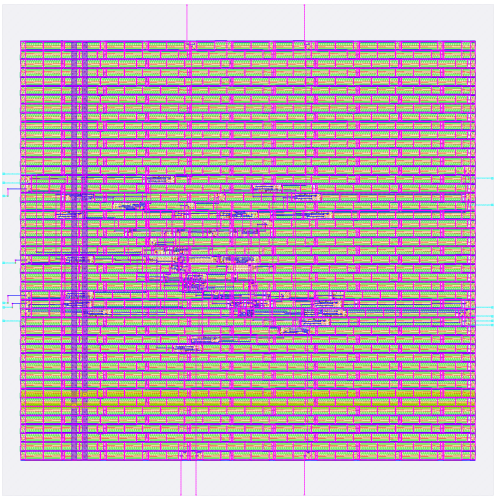
2) Critical Path and Comparison with $2N$ **Full-Adder Delays:** From `sta/mult_array_timing.rpt` (Table III), the critical path starts at input register `_106_` (`b_reg[0]`, `dfrtp_1`) and ends at output register `_100_` (`p[6]`, `dfrtp_1`), with a data arrival time of 2.471 ns and $T_{\min} = 2.604$ ns. Between these registers, the path traverses **8 combinational standard cells** (10 cells including the two flip-flops): **1 partial-product AND gate** (`_043_`, `and2_0` computing $\text{pp}_0[0]$) followed by **7 addition-network**

TABLE III: Ordered Cell-by-Cell Critical Path from `sta/mult_array_timing.rpt` ($T_{clk} = 2.60$ ns, `b_reg[0]` → `p[6]`).

#	Instance	Cell Type (`sky130_fd_sc_hd`)	Pin / Net	Delay (ns)	Time (ns)	Edge	Architectural Role in Multiplier
1	`_106_`	`dfrtp_1` (CLK→Q)	`b_reg[0]`	0.388	0.388	↓	Launch flip-flop for multiplier bit b_0
2	`_043_`	`and2_0` (B→X)	`pp0[0]`	0.180	0.568	↓	Partial-product AND gate: $\mathrm{pp}_0[0] = a_0 \wedge b_0$
3	`_056_`	`or3_1` (A→X)	`_012_`	0.391	0.959	↓	Adder network cell 1 (Row 1 carry/sum intermediate term)
4	`_058_`	`a32oi_1` (A3→Y)	`_014_`	0.250	1.208	↑	Adder network cell 2 (Row 1 → Row 2 coupling term)
5	`_072_`	`o21ba_1` (B1_N→X)	`_028_`	0.198	1.406	↓	Adder network cell 3 (Row 2 carry propagation term)
6	`_073_`	`xnor2_1` (B→Y)	`_029_`	0.147	1.553	↓	Adder network cell 4 (Row 2 sum → Row 3 col 1 input)
7	`_082_`	`maj3_1` (C→X)	`_037_`	0.385	1.937	↓	Adder network cell 5 (Row 3 col 1 majority carry-out $c_3[2]$)
8	`_084_`	`maj3_1` (B→X)	`_039_`	0.358	2.295	↓	Adder network cell 6 (Row 3 col 2 majority carry-out $c_3[3]$)
9	`_086_`	`xnor3_1` (C→X)	`row3.sum`	0.175	2.471	↓	Adder network cell 7 (Row 3 col 3 full-adder sum $s_3[3] = p_6$)
10	`_100_`	`dfrtp_1` (setup at D)	`p[6]`	−0.134	2.466	—	Capture flip-flop for product bit $p[6]$ (slack −0.004 ns at 2.60 ns)

TABLE IV: Record of All 8 LibreLane Place-and-Route Attempts (3.5 ns to 7.0 ns in 0.5 ns Increments).

Run	`CLOCK_PERIOD`	Setup (`max_ss`)	Setup (`nom_tt`)	Worst Hold	Wirelength	Cells	DRC/LVS	Signoff Outcome
1	3.5 ns	−3.034 ns	+0.340 ns	+0.114 ns	2,324 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
2	4.0 ns	−2.535 ns	+0.839 ns	+0.113 ns	2,328 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
3	4.5 ns	−2.035 ns	+1.339 ns	+0.113 ns	2,328 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
4	5.0 ns	−1.535 ns	+1.839 ns	+0.113 ns	2,328 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
5	5.5 ns	−1.035 ns	+2.339 ns	+0.113 ns	2,328 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
6	6.0 ns	−0.535 ns	+2.839 ns	+0.113 ns	2,328 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
7	6.5 ns	−0.035 ns	+3.339 ns	+0.113 ns	2,328 μm	1,883	0 / 0	FAIL (setup violation in slow corner `max_ss_100C_1v60`)
8	**7.0 ns**	**+0.465 ns**	**+3.839 ns**	**+0.113 ns**	**2,328 μm**	**1,883**	**0 / 0**	**PASS (signoff clean across all 9 PVT corners)**



(a) Whole-die view (150 μm × 150 μm): `VPWR`/`VGND` power grid (`met4`/`met5`), perimeter I/O pins (`met2`/`met3`), 47 standard-cell rows, active logic cluster, and tap/decap/filler array.



(b) Zoomed view of standard-cell rows and metal wires: placed `dfrtp_1`, `maj3_1`, and `xnor2_1` standard cells, local interconnect (`li1`), horizontal `met1`/`met3` and vertical `met2` tracks, and vias.

Fig. 3: KLayout screenshots of the final DRC- and LVS-clean layout `pnr/mult_array.gds` (Run 8 at $T_{clk} = 7.0$ ns).

cells (`_056_ or3_1`, `_058_ a32oi_1`, `_072_ o21ba_1`, `_073_ xnor2_1`, `_082_ maj3_1`, `_084_ maj3_1`, and `_086_ xnor3_1`). This closely matches the Section 2 estimate of $\approx 2N = 8$ full-adder delays: structurally, the longest

TABLE V: Part C Static Timing Analysis Results Table (`tt_025C_1v80`).

Metric	Value (with Units)
Worst slack at $T = 10$ ns (ns)	**+7.396 ns** (`7.40 ns` at 2 decimals)
$T_{\min}$ (ns)	**2.604 ns** (**2.60 ns** rounded to 0.01 ns)
$F_{\max}$ (MHz)	**384.02 MHz** (**384.62 MHz** at 2.60 ns)
Slack at $T = T_{\min}$ (ns)	**−0.004 ns** at 2.60 ns (`-0.00 ns`)
Critical path start point	`_106_` (`b_reg[0]`, `dfrtp_1`)
Critical path end point	`_100_` (`p[6]`, `dfrtp_1`)
Number of cells on critical path	**8 combinational cells** (1 AND + 7 adder cells; **10 cells** incl. launch/capture FFs)
Latency (ns) $= 2 \times T_{\min}$	**5.208 ns** (**5.20 ns** at 2.60 ns)
Throughput (million results/s)	**384.02 million results/s** (384.62 at 2.60 ns)

TABLE VI: Part D Physical Design Results Table (`pnr/metrics.json`).

Metric	Value (with Units)
`CLOCK_PERIOD` used (ns)	**7.0 ns** (from 3.5 ns in 0.5 ns steps)
Number of runs needed	**8 runs** (Run 8 passed all 9 corners)
Core area (μm^2)	**17,759.50** μm^2 (die area **22,500.00** μm^2)
Cells after synthesis (Part B)	**67 cells** (746.966 μm^2)
Cells after place and route	**1,883 total cells** (**328** stdcells + **1,555** fill/decap)
Kinds of cells added	• **Clock buffers** (`clkbuf_*`): **3 cells** (75.072 μm^2) • **Timing repair buffers** (`buf_*`): **19 cells** (105.101 μm^2) • **Tap cells** (`tapvpwrvgnd_1`): **245 cells** (306.544 μm^2) • **Fill / decap** (`fill_*`, `decap_*`): **1,555 cells**
Wirelength (μm)	**2,328** μm (with **530** vias across 93 nets)
Setup slack (ns)	**+0.465 ns** (`max_ss`); **+3.839 ns** (`nom_tt`)
Post-route $F_{\max}$ (MHz)	**153.01 MHz** (`max_ss`); **316.34 MHz** (`nom_tt`)
$F_{\max}$ from Part C (MHz)	**384.02 MHz** (pre-layout `tt_025C_1v80`)
DRC errors	**0** (Magic DRC = 0, KLayout DRC = 0)
LVS errors	**0** (Netgen LVS = 0, Antenna = 0)

path ripples through 1 AND gate, $N - 1 = 3$ vertical row transitions, and $N = 4$ horizontal stages in Row 3 ($1 + 7 = 8$ stages).

3) Flip-Flop Area Percentage (53.60%**):** The 16 `dfrtp_1` flip-flops occupy 400.384 μm^2 out of 746.966 μm^2, or **53.60%** of the total cell area (25.024 μm^2 per DFF vs. 346.582 μm^2 for all 51 combinational cells). Each master-slave edge-triggered D flip-flop with asynchronous reset contains two clocked latches, transmission gates, internal clock buffers, and reset transistors ($\sim$ 24–28 transistors, 6.67$\times$ the area of a 3.754 μm^2 `nand2_1` gate). Moreover, register count scales linearly as $4N$ ($2N$ input $+2N$ output FFs) while the combinational array scales quadratically as $\mathcal{O}(N^2)$ (N^2 AND gates $+N(N-1)$ adders); at $N = 4$, $4N = 16$ equals $N^2 = 16$, making flip-flop area dominant.

4) Post-Route $F_{\max}$ vs. Part C $F_{\max}$ and Added Physical Cells: Post-route $F_{\max}$ is **153.01 MHz** in the worst-case signoff corner (`max_ss_100C_1v60`, setup slack +0.465 ns at 7.0 ns) and **316.34 MHz** in the nominal corner (`nom_tt_025C_1v80`, setup slack +3.839 ns), compared to **384.02 MHz** in Part C. In the same nominal corner, $F_{\max}$ drops by 17.6% due to extracted RC parasitics across 2,328 μm of metal wiring and 530 vias, clock-tree skew (0.265 ns), and buffer insertion; multi-corner signoff is further limited by the slow-slow corner (1.60 V, 100 °C, 60.2% lower $F_{\max}$) where low supply voltage and high temperature reduce transistor drive current. While OpenROAD remapped the 51 combinational cells into 45 cells (keeping 16 DFFs), the flow increased total cell count from 67 to **1,883 cells** by adding: (1) **3 clock buffers** (`clkbuf_*`) during CTS to balance clock arrival across all 16 flip-flops; (2) **19 timing/slew repair buffers** (`buf_*`, `clkbuf_*`, `dlygate*`) to buffer I/O nets and fix max-slew/hold constraints; (3) **245 well/substrate tap cells** (`tapvpwrvgnd_1`) to bias the p-substrate to `VGND` and n-well to `VPWR` against CMOS latch-up; and (4) **1,555 filler and decoupling capacitor cells** (`fill_*`, `decap_*`) to maintain continuous n-well/power rails for DRC compliance and suppress dynamic IR drop.

5) DRC and LVS Verification and Manufacturing Signoff: Design Rule Checking (DRC) (Magic and KLayout: **0 errors**) verifies that all mask polygons obey the SkyWater 130 nm geometric and process rules (minimum metal width, spacing, via enclosure, well overlap, and antenna rules). **Layout Versus Schematic (LVS)** (Netgen: **0 errors**) extracts the physical transistor/cell netlist from the layout and compares it against the synthesized netlist (`synth/mult_array_netlist.v`) to confirm identical devices, nets, and pins. Both must pass before fabrication because DRC violations cause physical manufacturing defects (shorts, opens, oxide breakdown), whereas LVS violations mean a physically manufacturable chip implements the wrong circuit connectivity.

6) Scaling from a 4-Bit to an 8-Bit Array Multiplier: Using the Section 2 formulas ($\mathrm{FA}(N) = N(N-1)$ full adders, N^2 AND gates, and critical path $\approx 2N$ full-adder delays):

- **Full Adders**: At $N = 4$, $\mathrm{FA}_4 = 4(3) = 12$ full adders (16 AND gates). At $N = 8$, $\mathrm{FA}_8 = 8(7) =$ **56** full adders (64 AND gates), an increase of $56 - 12 = +$**44** full adders or $\frac{56}{12} =$ **4.67**$\times$ (quadratic $\mathcal{O}(N^2)$ growth).
- **Critical Path Delay**: Using $\approx 2N$ full-adder delays, the critical path grows from $2(4) = 8$ to $2(8) =$ **16** full-adder delays (2.0$\times$ **increase**, linear $\mathcal{O}(N)$ growth). Scaling from our Part C pre-layout measurement ($T_{\min,4} =$ 2.604 ns, with 2.083 ns combinational delay across 1 AND $+7$ adder stages ≈ 0.272 ns/FA), doubling the path gives $T_{\min,8} \approx 2 \times 2.604 =$ **5.21** ns ($F_{\max,8} \approx$ **192** MHz), or using $(2N-1) = 15$ full-adder stages plus 1 AND gate and flip-flop overhead: $T_{\min,8} \approx 0.388 + 0.180 + 15(0.272) - 0.134 =$ **4.51** ns ($F_{\max,8} \approx$ **221.6** MHz).

VII. Conclusion

A 4-bit unsigned registered array multiplier (`mult_array`) was designed structurally in Verilog using 16 AND gates, 12 full adders, and 16 flip-flops, verified across all 256 vectors in RTL and gate-level simulation (`PASS: 256 vectors checked, 0 errors`) alongside a 95-error fault-injection test, synthesized with Yosys to 67 `sky130_fd_sc_hd` cells (746.966 μm^2, 53.60% sequential area), analyzed in OpenSTA ($T_{\min} = 2.604$ ns, $F_{\max} = 384.02$ MHz), and implemented

through 8 LibreLane/OpenROAD runs into a DRC- and LVS-clean $150\,\mu\text{m} \times 150\,\mu\text{m}$ GDSII layout achieving 153.01 MHz multi-corner signoff operation (316.34 MHz nominal).

APPENDIX

A. Complete Simulation Logs (Passed Run, GLS, and 95-Error Injection)

1. Part A Behavioral RTL Pass Log & Part B GLS Pass Log

```
# Part A Behavioral RTL Simulation (vvp sim)
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:75: $finish called at 5146000 (1ps)

# Part B Gate-Level Simulation (vvp gls)
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:77: $finish called at 5147000 (1ps)
```

2. Part A Task 6 Error-Injection Log (`sum = a ^ b` — All 95 Mismatches)

```
VCD info: dumpfile mult.vcd opened for output.
a=3  b=3  exp=9   got=5   | a=11 b=6  exp=66  got=122
a=3  b=6  exp=18  got=10  | a=11 b=7  exp=77  got=81
a=3  b=7  exp=21  got=9   | a=11 b=9  exp=99  got=83
a=3  b=11 exp=33  got=29  | a=11 b=10 exp=110 got=78
a=3  b=12 exp=36  got=20  | a=11 b=11 exp=121 got=229
a=3  b=13 exp=39  got=23  | a=11 b=12 exp=132 got=244
a=3  b=14 exp=42  got=18  | a=11 b=13 exp=143 got=127
a=3  b=15 exp=45  got=17  | a=11 b=14 exp=154 got=162
a=5  b=5  exp=25  got=17  | a=11 b=15 exp=165 got=137
a=5  b=7  exp=35  got=27  | a=12 b=3  exp=36  got=52
a=5  b=10 exp=50  got=34  | a=12 b=6  exp=72  got=104
a=5  b=11 exp=55  got=39  | a=12 b=7  exp=84  got=68
a=5  b=13 exp=65  got=57  | a=12 b=11 exp=132 got=212
a=5  b=14 exp=70  got=54  | a=12 b=12 exp=144 got=208
a=5  b=15 exp=75  got=51  | a=12 b=13 exp=156 got=220
a=6  b=3  exp=18  got=10  | a=12 b=14 exp=168 got=136
a=6  b=6  exp=36  got=20  | a=12 b=15 exp=180 got=164
a=6  b=7  exp=42  got=18  | a=13 b=3  exp=39  got=55
a=6  b=11 exp=66  got=58  | a=13 b=5  exp=65  got=121
a=6  b=12 exp=72  got=40  | a=13 b=6  exp=78  got=110
a=6  b=13 exp=78  got=46  | a=13 b=7  exp=91  got=67
a=6  b=14 exp=84  got=36  | a=13 b=9  exp=117 got=101
a=6  b=15 exp=90  got=34  | a=13 b=10 exp=130 got=242
a=7  b=3  exp=21  got=9   | a=13 b=11 exp=143 got=223
a=7  b=5  exp=35  got=27  | a=13 b=12 exp=156 got=220
a=7  b=6  exp=42  got=18  | a=13 b=13 exp=169 got=145
a=7  b=7  exp=49  got=21  | a=13 b=14 exp=182 got=134
a=7  b=10 exp=70  got=54  | a=13 b=15 exp=195 got=171
a=7  b=11 exp=77  got=49  | a=14 b=3  exp=42  got=50
a=7  b=12 exp=84  got=36  | a=14 b=5  exp=70  got=118
a=7  b=13 exp=91  got=35  | a=14 b=6  exp=84  got=100
a=7  b=14 exp=98  got=42  | a=14 b=7  exp=98  got=74
a=7  b=15 exp=105 got=45  | a=14 b=10 exp=140 got=236
a=9  b=9  exp=81  got=65  | a=14 b=11 exp=154 got=194
a=9  b=11 exp=99  got=83  | a=14 b=12 exp=168 got=200
a=9  b=13 exp=117 got=101 | a=14 b=13 exp=182 got=134
a=9  b=15 exp=135 got=247 | a=14 b=14 exp=196 got=148
a=10 b=5  exp=50  got=34  | a=14 b=15 exp=210 got=186
a=10 b=7  exp=70  got=118 | a=15 b=3  exp=45  got=49
a=10 b=10 exp=100 got=68  | a=15 b=5  exp=75  got=115
a=10 b=11 exp=110 got=78  | a=15 b=6  exp=90  got=98
a=10 b=13 exp=130 got=114 | a=15 b=7  exp=105 got=77
a=10 b=14 exp=140 got=236 | a=15 b=9  exp=135 got=247
a=10 b=15 exp=150 got=166 | a=15 b=10 exp=150 got=230
a=11 b=3  exp=33  got=61  | a=15 b=11 exp=165 got=201
a=11 b=5  exp=55  got=39  | a=15 b=12 exp=180 got=196
a=15 b=13 exp=195 got=139 | a=15 b=14 exp=210 got=154
a=15 b=15 exp=225 got=181 |
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:75: $finish called at 5146000 (1ps)
```

B. OpenSTA Critical Path Timing Report (`sta/mult_array_timing.rpt`)

OpenSTA Critical Path Report ($T_{\text{clk}} = 2.60$ ns and 10.00 ns)

```
Startpoint: rst_n (input port clocked by clk) -> Endpoint: _094_ (recovery)
Path Group: asynchronous | Path Type: max | Arrival: 1.000 ns
Required: 2.828 ns (at 2.60 ns) -> Slack: +1.828 ns (MET) [+9.228 ns at 10.0 ns]

Startpoint: _106_ (rising edge-triggered flip-flop clocked by clk, b_reg[0])
Endpoint:   _100_ (rising edge-triggered flip-flop clocked by clk, p[6])
Path Group: clk   | Path Type: max
   Delay     Time   Description
---------------------------------------------------------
   0.000    0.000 ^ _106_/CLK (sky130_fd_sc_hd__dfrtp_1)
   0.388    0.388 v _106_/Q   (sky130_fd_sc_hd__dfrtp_1)
   0.180    0.568 v _043_/X   (sky130_fd_sc_hd__and2_0)
   0.391    0.959 v _056_/X   (sky130_fd_sc_hd__or3_1)
   0.250    1.208 ^ _058_/Y   (sky130_fd_sc_hd__a32oi_1)
   0.198    1.406 v _072_/X   (sky130_fd_sc_hd__o21ba_1)
   0.147    1.553 v _073_/Y   (sky130_fd_sc_hd__xnor2_1)
   0.385    1.937 v _082_/X   (sky130_fd_sc_hd__maj3_1)
   0.358    2.295 v _084_/X   (sky130_fd_sc_hd__maj3_1)
   0.175    2.471 v _086_/X   (sky130_fd_sc_hd__xnor3_1)
   0.000    2.471 v _100_/D   (sky130_fd_sc_hd__dfrtp_1)
            2.471   data arrival time
   2.600    2.600 ^ _100_/CLK (clock clk rise edge; 10.000 ns at T=10.0 ns)
  -0.134    2.466   library setup time (9.866 ns required at T=10.0 ns)
---------------------------------------------------------
           -0.004   slack at T = 2.60 ns (+7.396 ns slack MET at T = 10.00 ns)
```